MRIDANGA SEVA · PHASE 3 · FOR CODE MAINTAINERS (SEVA)

Face scan, explained

How a student checks in to class by looking at their own phone: what happens on the phone, on our server and at Google; which file does what; how privacy is protected; and what we learned from the first real tests on 24 October 2026.

Written for beginners and college students. No face-recognition background needed.
Branch `face-scan`, 25-10-2026.

Contents

1. What face scan is (and what it is not)
2. Four ideas you need first: detection, embedding, similarity, liveness
3. The big picture: phone, server, Google
4. Flow A: setting up a face at the desk (enrolment)
5. Flow B: checking in and out by face (self-scan)
6. The AI models and the native building blocks
7. The server side: tables, functions, the switch
8. Privacy and the law: what is kept, where, for how long
9. Code map: which file does what
10. The TEST app, builds and updates
11. Lessons from the first tests (three real bugs)
12. How to work on it safely, and a glossary

> **One sentence summary.** The student's own phone checks that the person in front of it is alive and is the same person whose "face code" it stored at the desk, reads its location once, gets a "this phone and app are genuine" pass from Google, and asks our server to record the check-in; the server checks everything again before it saves.

1. What face scan is (and what it is not)

Today a student checks in at the class door by showing the QR code in the app (or a printed QR card) to a coordinator. Face scan adds a second, optional way: the student taps **Check in** on their own phone, looks at the camera, and is checked in, but only **at the class**, only **during class hours**, and only if the face is **theirs**.

It is

- **Optional** and separate: its own written consent. QR stays for everyone, for good.
- **1:1** ("is this the owner of this login?"), never 1:N ("who is this among everyone?").
- **On-device**: the face is turned into numbers on the phone; no photo leaves it.
- Set up **once at the desk**, with a coordinator confirming it is the right person.

It is not

- A camera at the door, a tablet, or a scan of the whole room.
- Location tracking: location is read **once**, at the tap, and only "inside / outside + distance" is kept.
- A photo store: no picture is ever saved, on the phone or the server.
- Switched on for the class yet: it runs only in the separate TEST app, on the TEST server.

Why so careful?

A face is **biometric** data: you cannot change it like a password. India's Digital Personal Data Protection Act 2023 (DPDP) and its Rules 2025 ask for clear consent, keeping only what is needed, deleting it when done, and extra care for children. Our design follows that strictly: adults first, a separate consent form, the face code only on the student's phone, and everything deleted at the end of a test.

Who decided what (the decisions log)

Every rule has a numbered entry in `docs/DECISIONS.md`. The important ones for face scan: **#212** no location tracking · **#213** face is a separate opt-in, QR stays · **#248** self-scan on the student's own phone only · **#295** "Gate 2": no face code on the server · **#296** "Gate 1": the Google integrity check · **#344–#349** self-scan rules · **#430–#437** the TEST app and this week's fixes.

2. Four ideas you need first

Idea 1: face *detection* (where is a face?)

Detection finds faces in a camera frame and gives their box and a few landmarks (eyes, nose, mouth corners), the head angles (turned left/right = *yaw*), and how open each eye is (0 to 1). It does **not** know who the person is. We use Google's **ML Kit** face detector, which runs on the phone.

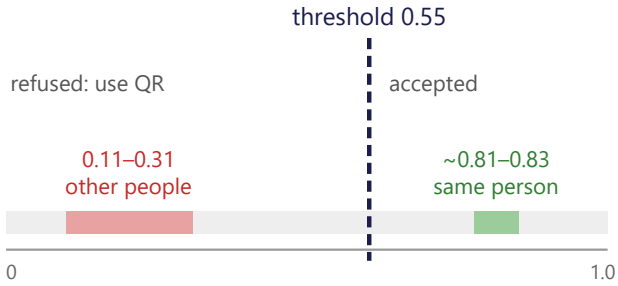
Idea 2: an *embedding* (a "face code")

A recognition model (**SFace**) looks at a cropped, straightened face (112 x 112 pixels) and outputs a list of **128 numbers**. Pictures of the same person give similar lists; different people give different lists. That list is the **face code**. You cannot turn it back into a photo. At the desk we take **5** pictures and store the **average** of their codes.

Idea 3: *cosine similarity* (how alike are two codes?)

Think of each code as an arrow in 128-dimensional space. Cosine similarity measures the angle between two arrows: **1.0** = same direction (same face), **0** = unrelated. In code: `cosine(a, b)` in `src/face/face-math.ts`.

Our first real test: Arjun scored **0.833** against his own code. In the October spike, the same person scored about 0.81 and different people 0.11–0.31.



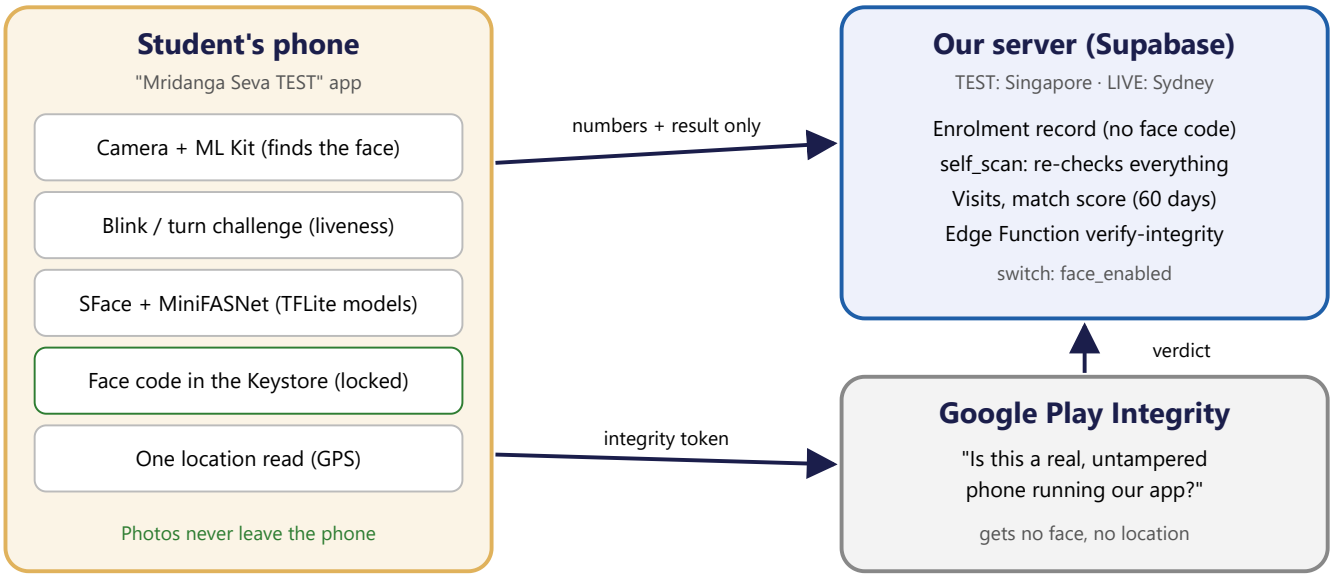
Thresholds: two kinds of mistakes

Mistake	What happens	How bad	Our choice
False accept	Someone else is checked in as you	Serious: wrong attendance	Threshold **0.55** (strict, setting `face_match_percent` = 55). A refusal only means "use your QR or ask a coordinator".
False reject	You are refused, even though it is you	Annoying, harmless	

Idea 4: *liveness* (is it a live person, not a photo?)

Two layers. **Active**: the app asks the person to **blink** and **turn their head**, in a random order (`src/face/liveness.ts`). A printed photo cannot do either. **Passive**: two small models (**MiniFASNet**) look at each picture for signs of paper or a screen and give a "real face" score; every picture must reach **0.8**.

3. The big picture: phone, server, Google



The two "gates" (security rules from the October audit)

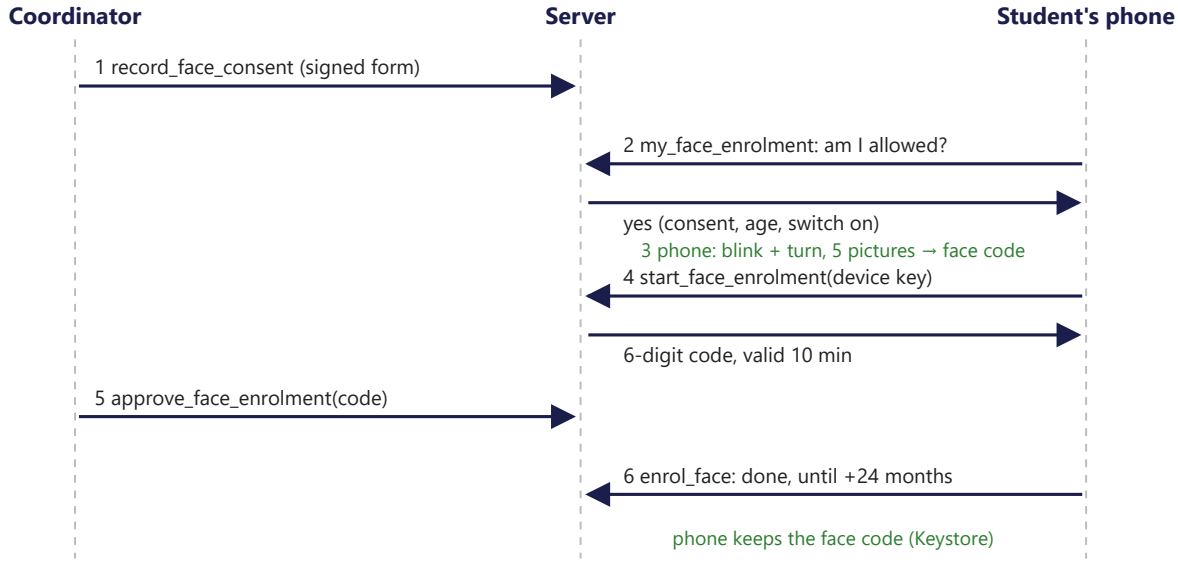
Gate 2: no face code on the server (#295). The server only keeps an *enrolment record*: who, which phone (a random device key), which model, who confirmed it, when, until when. If the server were ever leaked, there is nothing face-like to steal. A new phone = set up again at the desk.

Gate 1: the phone must prove it is genuine (#296). Before every self-scan the server hands out a one-time challenge; Google's Play Integrity signs a verdict about the phone and app; our Edge Function checks it and gives a one-time pass valid 120 seconds. A rooted phone, an emulator or a modified app gets no pass.

Why check things twice (phone and server)? The phone does the face work because the face must not leave it; but a phone can be tampered with, so the **server never trusts the phone's word alone**: it re-checks the integrity pass, the consent, the enrolment of *this* phone, the location result, the class hours, the liveness flag and the score.

4. Flow A: setting up a face at the desk (enrolment)

Happens once per student (adults: valid 24 months; children later: 12 months). Two people and two phones: the **student** and a **coordinator**.



1. **Consent.** The coordinator opens the student's profile → *Face scan at the desk* and records the signed form (`record_face_consent`). Nobody can record their own; under the minimum age (18 now) it refuses.
2. **Allowed?** The student's phone (My profile → Face scan) asks the server (`my_face_enrolment`). No consent or switch off → the camera never opens.
3. **Scan.** Blink and turn in random order; then 5 pictures 350 ms apart. Each picture: ML Kit finds the face again, it is cut and straightened, SFace gives 128 numbers, MiniFASNet gives a real-face score. Checks: at least 3 good pictures, every score ≥ 0.8, and every picture close to their average (≥ 0.5, "all one face"). The average is the face code.
4. **Ask.** The phone sends only its random **device key** and the model name (`start_face_enrolment`); the server answers a **6-digit code** (10 minutes). The phone shows it.
5. **Confirm.** A coordinator, on their own phone, types the code (`approve_face_enrolment`). This is the "supervised" part: a person checks it is the right student. 5 wrong codes end the request; you can never confirm your own.
6. **Finish.** The student's phone notices (it asks every 4 s), calls `enrol_face`, and keeps the face code in Android's **Keystore**-encrypted storage (`expo-secure-store`). The server records "enrolled until …".

Screens: `app/src/app/student/face-scan.tsx` (student), `app/src/app/staff/face/[id].tsx` (desk). Logic: `src/face/enrolment.ts`, `src/face/face-scanner.tsx`, `src/data/face.ts`.

5. Flow B: checking in and out by face (self-scan)

The home screen shows a **Check in** circle only when: face scan is switched on, this phone holds a confirmed face code, and the class centre is open now (`self_scan_state`). After a check-in it shows **Check out**.

#	Step	Where	Refused if...
1	**One location read**, in the foreground, at most 8 s, high accuracy	phone: `self-scan-location.ts`	outside the area (radius + GPS accuracy), accuracy worse than 100 m, a "mock location" app, no position
2	**Face**: blink + turn, then 3 pictures; each a real face (≥ 0.8), all one face, average close to the stored code (≥ 0.55)	phone: `self-scan-rules.ts` `verifyScan`	not live, more than one face, no match
3	**Integrity**: challenge → Play Integrity token bound to (login, action, challenge, location result) → `verify-integrity` → 120 s pass	phone: `integrity.ts` ; server: Edge Function	rooted / emulator / modified app, wrong package or signing key, old challenge
4	**self_scan**: server re-checks pass, switch, consent, this phone's enrolment, location, hours, liveness, score; saves the visit (method *face*, by the student)	server: migration `0071`	any of the above; outside class hours; checked in less than 10 min ago (check-out)

> **Check-out is never blocked by place.** If the phone is outside the area at check-out, the check-out is **saved and marked "away"** for the staff (`visits.check_out_location`). A parent of a child gets the usual check-out email (from the existing parent-notice feature).

> **Three failed tries in a day** → the app only offers the coordinator's way (QR card or roll number). Every step shows "Or ask a coordinator". Failures are counted per centre and reason, **without the student's name** (`face_scan_failures`), for the staff page *Face scan use*.

What a check-in record looks like (first real one, 24-10-2026)

check_in	method	liveness_ok	location	match score	marked by
20:05:07	face	true	inside, 7 m	0.833 (kept 60 days)	the student

6. The AI models and the native building blocks

Models (all run on the phone, free, open licences)

Model	Job	Input → output	File / size
Google ML Kit face detection	find faces, landmarks, eyes open, head angles	camera frame → boxes + points	inside the app library
SFace (OpenCV Zoo), int8	face code (recognition)	112×112 aligned face → 128 numbers	`sface_2021dec_int8.tflite`, 9.9 MB
MiniFASNet V2 (x2.7 crop)	passive liveness	80×80 crop → real / fake	`minifasnet_v2_2_7_int8.tflite`, 0.6 MB
MiniFASNet V1SE (x4.0 crop)	passive liveness, second view	80×80 wider crop → real / fake	`minifasnet_v1se_4_0_int8.tflite`, 0.6 MB

"int8" = the model's weights are stored as small integers (smaller, faster on cheap phones). The model name is also a server setting (`face_model_version`): changing the model ends every enrolment, and everyone sets up again.

The picture pipeline (`src/face/pipeline.ts`)

1. The camera hands a frame (VisionCamera) → ML Kit finds the face on the camera thread.
2. When a picture is wanted, the frame becomes an image (Nitro Image), saved to a temporary JPEG, ML Kit runs again on it for exact landmarks, the file is **deleted in the same call**.
3. The face is rotated and scaled so the eyes, nose and mouth sit where SFace expects (`alignedFaceTensor` in `face-math.ts`); two looser crops go to the two MiniFASNets.
4. TensorFlow Lite runs the three models; out come the 128 numbers and the real-face score.

Native libraries (why the app needs its own APK build)

Library	What it does for us
`react-native-vision-camera` 5 + `-worklets`	front camera; runs code on each frame on a separate thread ("worklets"), so the screen stays smooth
`react-native-vision-camera-face-detector`	ML Kit face detection inside those frame worklets
`react-native-fast-tflite`	runs the .tflite models (TensorFlow Lite / LiteRT)
`react-native-nitro-modules`, `-nitro-image`	fast bridge between JavaScript and native code; images in memory
`expo-secure-store`	stores the face code encrypted with a key held by Android's Keystore
`@expo/app-integrity`	asks Google Play Integrity for a signed verdict
`expo-location` (already in the app)	the one location read

> Anything in this table is **native** code: adding or changing it changes the app's **fingerprint** and needs a new APK (section 10). Changes to our own TypeScript can reach phones as an over-the-air **update** instead.

7. The server side: tables, functions, the switch

Our backend is **Supabase** (PostgreSQL + login + small server functions). All face rules live in two SQL migrations on branch `face-scan`: `supabase/migrations/0070_face_enrolment.sql` (setup, consent, integrity) and `0071_face_selfscan.sql` (self check-in/out). The numbers 0070/0071 are placeholders, renumbered when face scan joins the main app.

Tables

Table	Holds	Kept
`face_enrolments`	who, device key, model, who confirmed, dates, how it ended. **No face code.**	until ended, then 400 days as proof
`consents` (scope *face*)	the written consent; for a child also the parent and the child's own yes	as the class's consent rules
`visits` + `visit_face_scores`	the check-in/out; liveness ok; check-out location result; the match score	score 60 days
`integrity_challenges`	one-time challenges and passes (hash only)	1 day
`face_scan_failures`	centre, reason, time. **No student id.**	400 days

Functions the app calls (each checks who is calling)

Function	Who	Does
`record_face_consent`	staff	records the signed form (not for your own record)
`my_face_enrolment`	student	is this phone's enrolment current?
`start_face_enrolment` / `enrol_face`	student	ask for the 6-digit code / finish after confirmation
`approve_face_enrolment`	staff	type the code (never your own)
`remove_face`	student or staff	"Turn face scan off": ends the enrolment (and the consent if the person says no)
`issue_integrity_challenge`	student	one-time challenge, 60 s
`self_scan_state` / `self_scan`	student (or staff with an own record)	home circle / the check-in or check-out
`face_use_counts`	staff	counts for the *Face scan use* page

The Edge Function `supabase/functions/verify-integrity` is the only caller of `begin_integrity_check` / `finish_integrity_check`; it holds the Google service-account key as a secret.

Settings (changed in the SQL editor only, never from the app)

Setting	Value now	Meaning
`face_enabled`	false on LIVE · true on TEST for the test	the master switch: off = every face entry hidden
`face_min_age`	18	adults only until the Guru decides (recommended 13)
`face_match_percent`	55	the 1:1 threshold (cosine 0.55)

`face_expiry_months_adult / _minor`	24 / 12	set up again after this
`face_score_days`	60	match scores deleted after this

A nightly job `mridanga-face-housekeeping` ends expired and unfinished enrolments and deletes old scores, challenges and failure counts. A coordinator who is also a student (linked "own record") can use face scan too; only the self-scan may record their own visit (#434).

8. Privacy and the law

Data	Where	How long
Photos / video	nowhere (memory only for a second; a temporary file deleted at once)	never kept
Face code (128 numbers)	only the student's phone, Keystore-encrypted	until turned off, sign-out, expiry or app removed
Enrolment record	server (TEST: Singapore; LIVE: Sydney, Australia)	until ended, then 400 days as proof
Location	only "inside / outside + distance" for each check-in / check-out	with the visit
Match score	server	60 days
Integrity verdict	Google sees the phone/app check, not the face or location	-

Rules we follow

- **Consent first**, separate form, nothing pre-ticked; saying no changes nothing (QR stays). The person can stop any time (Turn face scan off): the code is wiped and the server record ends.
- **Location once** (#212): only at the tap; never in the background, never a map, never in a parent email.
- **Children only later**: written, verifiable parent consent (DPDP Rules, rule 10), the child's own yes, minimum age decided by the Guru, and only in the real class app after the full release (`docs/phase3/ADULT_TEST.md` §3).
- **Data outside India**: our server is in Australia (LIVE) and Singapore (TEST); the DPDP Act allows this unless a country is restricted, and the privacy notice says so.
- **Test data is deleted** within 7 days of the last test session: phones (turn off, uninstall) and one SQL run on TEST with a check query (`docs/OPERATIONS.md` "Face scan: deleting the test face data").

> **For maintainers:** never add a column that stores a face code on the server, never send pictures anywhere, never read location in the background, never log faces or scores with names. The smoke tests check several of these (e.g. "no column named template or embedding").

9. Code map: which file does what

```
app/
  app.config.js              the "Mridanga Seva TEST" variant (APP_VARIANT=facetest); unchanged app otherwise
  eas.json                   build profiles: preview, production, facetest
  assets/models/             the three .tflite models + README (licences, checksums)
  src/face/
    face-camera.tsx          VisionCamera + ML Kit on every frame; picture capture
    face-scanner.tsx         one scan: permission, blink/turn, pictures, errors (used by both flows)
    face-guide.tsx           the oval guide, sweeping light and progress ring over the camera (#437)
    face-guide-state.ts      the guide's rules (pure, tested)
    liveness.ts              the blink/turn state machine (pure, tested)
    pipeline.ts              picture → SFace code + MiniFASNet score; loads the models (#436)
    face-math.ts             alignment, crops, cosine, averages (pure, tested)
    enrolment.ts             what the phone keeps, the wipe rule, quality checks (pure, tested)
    face-store.ts            Keystore-encrypted storage of the face code
    face-status.ts           asks the server at start; applies the wipe rule
    self-scan-rules.ts       verifyScan (1:1 decision), location rules (pure, tested)
    self-scan-location.ts    the one location read
    integrity.ts             Play Integrity token (Android only)
    face-settings.ts         all numbers: frames, thresholds, timeouts, Cloud project number
  src/data/face.ts, self-scan.ts      the server calls
  src/app/student/face-scan.tsx       S15 set up / turn off
  src/app/student/self-scan.tsx       S16 check in / check out
  src/app/staff/face/[id].tsx         C25 Face scan at the desk
  src/app/staff/face-counts.tsx       C27 Face scan use (counts)
  src/app/staff/face-dev.tsx          C26 hidden developer screen (TEST builds only)
  src/app/staff/me/face-scan.tsx, self-scan.tsx   the same for a coordinator's own record
  tests/face-scan, face-enrolment, self-scan, app-variant, build-guard .test.mjs
supabase/
  migrations/0070_face_enrolment.sql, 0071_face_selfscan.sql
  functions/verify-integrity/          the Edge Function (verdict rules in verdict.ts)
  tests/smoke-test.mjs                 2,126 checks incl. every face rule and the deletion runbook
docs/
  DECISIONS.md (#295-#300, #307-#312, #344-#349, #430-#437), DATABASE.md, OPERATIONS.md, SCREENS.md
  phase3/ ADULT_TEST.md, FACE_ATTENDANCE_PLAN.md, DPIA_DRAFT.md, CONSENT_DRAFT.md, print/
```

"Pure, tested" files have no React Native imports, so the unit tests run them in plain Node with made-up camera frames. Start reading at `face-scanner.tsx` and follow the imports.

10. The TEST app, builds and updates

Why a separate app

Testers keep the real "Mridanga Seva" app (their real data) and install **"Mridanga Seva TEST"** next to it: a different package name `org.mridangaseva.app.test`, a red TEST band on the icon, the TEST server only. One line in `app.config.js` does it: when `APP_VARIANT=facetest`, the name, package, link scheme and icons change; when it is not set, the app config is **byte-for-byte** the real app's (we proved this, #432). A "build guard" script refuses any TEST-app build pointed at LIVE.

Fingerprint, builds and updates (the most important idea for releases)

> **Build (APK)** = the whole app including native code. Made on Expo's servers (EAS Build), about 25 minutes, 15 free per month. Needed when native code or config changes (a new library, app.json, eas.json).

> **Update (OTA)** = only our JavaScript, downloaded by the installed app at start. Minutes, unlimited. It reaches only APKs with the same **fingerprint**.

The **fingerprint** is a hash Expo computes from everything native (packages, app.json, eas.json, plugins). Same fingerprint = an update fits. The class's phones run fingerprint `23dc7e4a`; the face branch has native face libraries, so its fingerprint differs. That is why face scan stays on its own branch until a new production APK is planned: merging earlier would stop updates reaching the class's phones.

App	Channel	Fingerprint	Talks to
Class app (production)	production	`23dc7e4a`	LIVE
Mridanga Seva TEST (build 560780b7)	facetest	`fde0a96f`	TEST

Publishing a TEST-app update: `$env:APP_VARIANT='facetest'` then `npx eas-cli update --channel facetest --environment preview --platform android -m "…"` (OPERATIONS.md). Office network: unplug the cable and use a phone connection; the office firewall breaks uploads.

11. Lessons from the first tests (three real bugs)

All three were invisible in development and in automatic checks, and appeared only on a real phone. Each fix also added a check so it cannot come back.

Bug 1: the app closed the moment it opened (#435)

Cause. To save space, an earlier change removed a GPU library (`libtensorflowlite_gpu_jni.so`) from the APK. But the TFLite library is *linked* to it and Android loads it at app start: a missing linked library = instant crash. The automatic check only *compiled* the APK, it never opened it.

Fix. Keep the library. **New check:** CI reads every native library in the APK (`readelf`) and fails if one needs a library that is missing. **Lesson:** "it builds" is not "it runs".

Bug 2: stuck at "picture 0 of 5" (#436)

Cause. In an installed app, bundled files are Android resources with plain names like `assets_models_sface_2021dec_int8`; the model loader expected a web-style address and failed with "*MalformedURLException: no protocol*". In development the files came from the dev server as `http://` addresses, so it always worked there. Worse, the screen **swallowed the error** and kept waiting.

Fix. Copy each model to the app's cache with `expo-asset` and load the `file://` copy. The scanner now stops after 3 failed pictures and says so. We found the real error with the hidden **developer screen**, which prints it. **Lessons:** test the release build, and never hide an error without telling the user.

Bug 3: uploads cut off at 9 MB

Cause. The PC had the office cable and the phone's USB tethering both on; Windows preferred the office network, whose firewall breaks large uploads to Google's storage. **Fix:** unplug the office cable during builds and updates. **Lesson:** when something fails half-way, check the network path, not only the code.

> **And what worked first time:** the 1:1 match (score 0.833), the blink/turn challenge, Google Play Integrity through our own Edge Function, the server's re-checks, the 10-minute rule, the "outside the area" refusal, and the "two faces in view" refusal.

12. How to work on it safely

1. **Never LIVE.** Work only with the TEST server (`fhuqyk…`) and the TEST app. The main folder's `app/.env` is LIVE: use a worktree with TEST values (`app/.env.test` copied as `.env.development`).
2. **Branch `face-scan`, never `main`**, until the release lead merges it before a production APK.
3. **Before every commit**, from `app/`: `npx tsc --noEmit` (0 errors), `npx expo lint` (clean), `npm test` (all pass); from `supabase/tests/`: `npm test` (smoke + replay).
4. **Native or JS?** Check the fingerprint (`npx expo-updates runtimeversion:resolve --platform android`, with `APP_VARIANT=facetest` for the test app). Unchanged → an update is enough.
5. **Write it down**: a numbered entry in `docs/DECISIONS.md` for every rule you add or change, and the docs it touches.
6. **Test on a phone** with the developer screen (TEST builds: staff My profile → *Face scan test (developer)*): it shows model load times, live face numbers, and the real error of any picture.

Glossary

1:1 / 1:N	compare with one stored person / search among many. We only do 1:1.
Embedding / face code	128 numbers describing a face; not reversible to a photo.
Cosine similarity	how alike two face codes are, 0 to 1.
Liveness	proof that a live person, not a photo or video, is in front of the camera.
Enrolment	setting up the face once, at the desk, confirmed by a coordinator.
Keystore	Android's locked key storage; our face code is encrypted with a key kept there.
Play Integrity	Google's signed verdict that the phone and app are genuine.
Edge Function	a small server program on Supabase (ours: `verify-integrity`).
Migration	a numbered SQL file that changes the database; run on TEST first, LIVE much later.
Fingerprint	a hash of the app's native parts; updates only reach APKs with the same one.
OTA update	new JavaScript sent to installed apps without a new APK.
DPDP	India's Digital Personal Data Protection Act 2023 and Rules 2025.

Questions about privacy: Regula Sarvesh Chandra, privacy@mridangaseva.com. Code: the face-scan branch on GitHub (Praviadari/mridanga-seva). This document: `docs/phase3/print/face-scan-explained.html`.